Session Types as Causal Timelines for Effect Ordering

The Prologos Project - from Zachary Larson

2026-03-06

Contents

1 1. The Central Thesis 3

2 2. Grounding: What We Have Today 4
2.1 2a. The Session Lattice 4
2.2 2b. Session Advancement 4
2.3 2c. Direct IO Execution 5

3 3. The Formal Connection: Sessions Encode Causal Order 5
3.1 3a. Curry-Howard: Propositions as Sessions 5
3.2 3b. Linear Logic's Tensor and Sequencing 6
3.3 3c. Causality in Linear Logic (Abramsky, Melliès) 6
3.4 3d. Session Fidelity = Causal Consistency 7

4 4. The Galois Connection: Session Lattice Effect Lattice 7
4.1 4a. What Is the Effect Lattice? 7
4.2 4b. The Galois Connection 8
4.3 4c. Monotonicity of and 9
4.4 4d. What the Galois Connection Enables 9

5 5. Architecture D: Session-Derived Effect Ordering 10
5.1 5a. Overview 10
5.2 5b. Session Advancement as Causal Clock 10
5.3 5c. Process Structure Determines the Partial Order 11
5.4 5d. The Effect Handler 12
5.5 5e. Comparison with Architectures A-C 12

6 6. Deeper Implications: The Parameterised Monad View 13
6.1 6a. Session Types as Indexed Pre/Post Conditions 13
6.2 6b. The Effect Quantale Connection 13
6.3 6c. The Category-Graded Monad Unification 14

7 7. Interaction with Our Existing Infrastructure 15
7.1 7a. The Cross-Domain Bridge Pattern 15
7.2 7b. Integration with compile-live-process 16
7.3 7c. The ATMS Connection for Speculative Effects 16

8 8. The Session-Effect Duality 17
8.1 8a. An Observation 17
8.2 8b. Implications for IO 17

9 9. Challenges and Open Questions 17
9.1 9a. Recursive Sessions and Infinite Timelines 17
9.2 9b. Choice and Non-Deterministic Sessions 18
9.3 9c. Data Dependencies Across Channels 18
9.4 9d. When Sessions Don't Cover All Effects 19

10 10. Discussion: Resolving the Open Questions 19
10.1 10a. Cross-Channel Data Dependencies Are Constraint Resolution (§9c Resolved) 19
10.2 10b. ATMS for Branching Effect Orders 20
10.3 10c. The `rel=/=proc` Structural Parallel 21
10.4 10d. Partial Order Planning Connection 22
10.5 10e. Architecture D Refined: Five Layers 22
10.6 10f. `main` as Implicit Session 23

11 11. Architecture Decision: A + D 24
11.1 11a. The Recommended Pair 24
11.2 11b. Why B Is Subsumed 24
11.3 11c. When C Might Be Relevant 25
11.4 11d. Layered Architecture 25

12 12. Formalization Roadmap 25
12.1 Phase D1: Effect Position Lattice (Small, Self-Contained) . . 25
12.2 Phase D2: Session-Effect Bridge Propagator 25
12.3 Phase D3: Effect Descriptor Cells 25
12.4 Phase D4: Effect Handler 26
12.5 Phase D5: Vector Clock for Multi-Channel 26

13 13. Critique and Limitations **26**
13.1 13a. Data-Flow Analysis Complexity 26
13.2 13b. Cycle Detection as Deadlock Detection 26
13.3 13c. Performance Considerations 27
13.4 13d. The "Fully Native" Limit 27

14 14. Conclusions **27**
14.1 14a. The Core Discovery 27
14.2 14b. The Galois Connection 28
14.3 14c. Architecture A + D 28
14.4 14d. The `rel=/=proc` Parallel 29
14.5 14e. Architecture D as Natural Extension 29

15 References **30**
15.1 Prologos Internal 30
15.2 Foundational 30
15.3 Causality and Linear Logic 30
15.4 Effect Systems and Graded Monads 31
15.5 Session Type Properties 31
15.6 Propagators, CRDTs, and Distributed Systems 31
15.7 Planning and Ordering 31

; Context: Effectful Propagators Research §9c — "Could session types serve as the causal context for IO effects?" ; Status: Research document — formal exploration with novel synthesis ; Prerequisite: ‘2026-03-06EFFECTFULPROPAGATORSRESEARCH.md ‘2026-02-24LOGICENGINEDESIGN.org‘

1 1. The Central Thesis

A session type is not merely a protocol specification — it is a *causal timeline*. Each position in a session type (each `!A . S` or `?A . S` node) represents a causally-ordered point in time. The session's syntactic structure *is* the happens-before relation for effects at that channel.

If we formalize this observation, session types can serve as the causal context that provides effect ordering on our monotone propagator substrate — not as an external scheduling discipline layered on top, but as an *intrinsic property* of the session lattice itself. The ordering doesn't need to be recovered; it was always there, encoded in the session type's continuation structure.

This potentially opens a *fourth architecture* beyond the three proposed in the effectful propagators research: one where effect ordering is derived

from the session lattice via a Galois connection to an effect lattice, rather than imposed by timestamps, barriers, or topological scheduling.

2 2. Grounding: What We Have Today

2.1 2a. The Session Lattice

Our session lattice (`session-lattice.rkt`) orders session types by information:

```
sess-bot ( - no information, fresh cell)
    ↓
sess-send(A, S) / sess-recv(A, S) / sess-choice(...) / ...
    ↓
sess-top ( - contradiction, incompatible protocols)
```

The merge function (`session-lattice-merge`) is a join: it unifies two session types if they're compatible, and produces `sess-top` if they conflict. This is a flat lattice at the top level — `sess-bot → concrete → sess-top` — but the internal structure of a concrete session is a *chain*:

```
!String . ?Int . end
   ↓        ↓      ↓
  pos 0   pos 1   pos 2
```

Each position in this chain is causally ordered: position 1 cannot occur until position 0 has been performed. This ordering is enforced by the continuation structure — to reach `?Int`, you must first decompose `!String . ?Int . end` into `!String` + `(?Int . end)`.

2.2 2b. Session Advancement

In `session-runtime.rkt`, session advancement works via propagators:

```
(rt-add-session-advance-in-rnet rnet
  src-cell       ;; current session cell
  dst-cell       ;; next session cell
  predicate      ;; e.g., sess-send-like?
  cont-extractor ;; extracts continuation from session
)
```

This installs a propagator that watches `src-cell` and, when it matches the predicate, extracts the continuation and writes it to `dst-cell`. The chain of advancement propagators mirrors the session type's structure exactly:

```
cell: !String . ?Int . end
  → advance propagator fires, writes to cell
cell: ?Int . end
  → advance propagator fires, writes to cell
cell: end
```

Each cell corresponds to one position in the session timeline. The advancement is monotone: `sess-bot → !String.?Int.end → end` (within each cell), and the chain of cells moves through the session positions sequentially.

2.3 2c. Direct IO Execution

Today, IO effects execute inline during `compile-live-process`, which walks the process AST recursively. The walk visits `proc-send`, `proc-recv`, `proc-open` in syntactic order — the same order the session type dictates.

```
[(proc-send expr chan cont)
 ;; Direct IO: if IO channel, write to file immediately
 (when io-cell
   (write-string str-val port)
   (flush-output port))
 ;; Advance session, then recurse into continuation
 (compile-live-process rnet3 cont ...)]
```

The session type determines the process structure, the process structure determines the AST walk order, and the walk order determines effect execution order. Session → Process → Walk → Effects. But this chain is mediated by the AST, not by the session lattice directly.

3 3. The Formal Connection: Sessions Encode Causal Order

3.1 3a. Curry-Howard: Propositions as Sessions

The foundational insight comes from Caires & Pfenning (2010) and Wadler (2012): there is a Curry-Howard correspondence between linear logic and session types.

Linear Logic	Session Types	Process Calculus
Proposition	Session type	Channel type
Proof	Process	Implementation
Cut	Channel composition	Communication
Cut reduction	Computation step	Message exchange

In this correspondence, *cut elimination steps correspond to process reductions*. When two processes communicate (cut), the proof simplifies (cut reduction), and the protocol advances. The ordering of cut elimination steps IS the ordering of communications.

Sources:

- Propositions as Sessions — Wadler (2012)
- Linear Logic Propositions as Session Types — Caires & Pfenning (2012)
- Session Types as Intuitionistic Linear Propositions — Caires & Pfenning (2010)

3.2 3b. Linear Logic's Tensor and Sequencing

In linear logic, the tensor () connective corresponds to parallel composition, and the "of course" (!) modality to reusable resources. But the key connective for sequencing is the *linear implication* (): `A  B` means "consume A to produce B." In session terms: receive A, then continue with B.

This is exactly our `sess-recv(A, S)` — consume an A from the channel, then continue with protocol S. The continuation structure of session types IS linear implication applied repeatedly:

```
?A . ?B . !C . end    A  (B  (C  1))
```

The sequencing comes from the nesting of implications. Each "consume" must happen before the next one is available. This is a *proof-theoretic* ordering, not a scheduling decision.

3.3 3c. Causality in Linear Logic (Abramsky, Melliès)

Research on causality in linear logic (Abramsky & Melliès) formalizes the dependency structure of proofs using event structures. In proof nets (the graph-theoretic presentation of linear logic), some inference rules can commute (their order doesn't matter) and some cannot. The rules that cannot commute have a causal dependency — one must happen before the other.

These causal dependencies form a partial order, and this partial order is exactly the partial order of events in the corresponding process. For session-typed processes, the causal dependencies are:

- *Sequential within a session*: `send x; recv y` — the send must precede the recv (they're on the same channel, in continuation order)
- *Independent across sessions*: operations on channel `a` are concurrent with operations on channel `b` (they're in separate proof branches)
- *Synchronized at joins*: `proc-par` with shared channels creates synchronization points

This is precisely the happens-before relation we need for effect ordering. *Sources*:

- Causality in Linear Logic — Hayman & Winskel
- Games and Full Completeness for MLL — Abramsky & Jagadeesan
- Sequentiality vs. Concurrency in Games and Logic — Melliès

3.4 3d. Session Fidelity = Causal Consistency

Session type systems guarantee *session fidelity*: "session communications follow the order prescribed by the protocol." This is not just a type safety property — it is a causal consistency guarantee. A well-typed session process respects the happens-before relation encoded in its session type.

Combined with type preservation (if `P` and `P → Q`, then `Q`) and progress (well-typed processes either terminate or can make a step), session types guarantee that effects happen in causal order and that all required effects eventually happen.

Sources:

- Session Fidelity and Deadlock Freedom
- Comparing Deadlock-Free Session Typed Processes

4 4. The Galois Connection: Session Lattice Effect Lattice

4.1 4a. What Is the Effect Lattice?

We need to define what the "effect lattice" is before we can connect it to sessions. For IO effects, we propose:

Effect Position Lattice (EPL): A lattice that tracks the causal position of an effect within a session timeline.

```
Elements:
  eff-bot              - no effect information
  eff-pos(session-path, depth)   - effect at a specific position in a session
  eff-top              - conflicting position information

where:
  session-path = sequence of session operations leading to this position
  depth = Nat - position index within the session chain
```

The ordering is flat within each session (positions don't refine each other) but the *depth* provides a total order on positions within a single session: `eff-pos(s, 0) < eff-pos(s, 1) < eff-pos(s, 2) < ...`

For multiple sessions (concurrent channels), positions form a partial order: positions on different session paths are incomparable (concurrent), while positions on the same path are totally ordered.

4.2 4b. The Galois Connection

We propose a Galois connection `(, )` between the session lattice and the effect position lattice:

```
 : Session → EffPos      (abstraction - extract effect position)
 : EffPos  → Session      (concretization - what session state corresponds to this posi

Adjunction: (s)  e    s  (e)
```

(Session → Effect Position): Given a session type at a cell, extract the current position in the effect timeline.

```
(sess-bot) = eff-bot                                  (no info → no position)
(!A . S)   = eff-pos(path, depth)  where depth = session chain depth
(?A . S)   = eff-pos(path, depth)  where depth = session chain depth
(end)      = eff-pos(path, max-depth)                 (terminal position)
(sess-top) = eff-top                                  (contradiction → conflict)
```

(Effect Position → Session): Given an effect position, reconstruct the remaining session protocol.

```
(eff-bot)            = sess-bot                        (no position → no session info)
(eff-pos(path, d))   = tail(original-session, d)       (session from position d onwar
(eff-top)            = sess-top                        (conflict → contradiction)
```

4.3 4c. Monotonicity of and

Both functions must be monotone for the Galois connection to hold:

is monotone: As the session cell refines (bot → concrete → advanced → end), the effect position increases monotonically. Session advancement never goes backward — once you've sent at position 0, you're at position 1 forever. This follows from the monotonicity of the session lattice merge.

is monotone: As the effect position increases, the remaining session "shrinks" (less protocol to execute). In the session lattice, this is an increase in information (from "full protocol remaining" to "less protocol remaining" to "end"). The concretization respects this ordering.

Adjunction: Knowing that an effect is at position `d` tells us at least as much as knowing the session is at position `d` (the session carries more information — what type to send/receive — but the position is the common abstraction).

4.4 4d. What the Galois Connection Enables

With this connection formalized, we get:

1. *Effect cells that derive ordering from session cells*: An effect cell doesn't need its own timestamp. Its position is computed from the session cell's value via . As the session advances monotonically, the effect position advances monotonically.

2. *Cross-domain propagators*: A bridge propagator between the session lattice and the effect lattice, analogous to the existing type/session bridge. When the session cell advances (e.g., `!String.?Int.end → ?Int.end`), the bridge propagator updates the effect cell (e.g., `eff-pos(ch, 0) → eff-pos(ch, 1)`).

3. *Soundness of effect ordering*: If the session type system is sound (session fidelity), then the derived effect ordering is sound. Effects at position `d` provably happen before effects at position `d+1`, because session advancement is provably monotone.

4. *Parallel effects fall out naturally*: Effects on different channels get incomparable positions (different session paths). The effect lattice's partial order correctly captures that these effects are concurrent and may execute in either order.

5 5. Architecture D: Session-Derived Effect Ordering

This section proposes a fourth architecture — one that the three architectures in the effectful propagators research don't cover, because it derives ordering from the session structure rather than imposing it externally.

5.1 5a. Overview

```
Session Lattice                    Effect Position Lattice
(session-lattice.rkt)              (NEW: effect-position.rkt)

 sess-bot             →  eff-bot
    ↓                                ↓
 !A . ?B . end        →  eff-pos(ch, 0)
    ↓ (advance)                      ↓
 ?B . end             →  eff-pos(ch, 1)
    ↓ (advance)                      ↓
 end                  →  eff-pos(ch, 2)
    ↓                                ↓
 sess-top             →  eff-top

        ↑


Bridge propagator: watches session cell, updates effect position cell via
Effect handler: watches effect position cells, executes effects in position order
```

5.2 5b. Session Advancement as Causal Clock

The key insight: *session advancement is a causal clock.*

In distributed systems, a Lamport clock increments at each event and each message send/receive. A vector clock maintains per-process counters. These provide causal ordering: if `clock(A) < clock(B)`, then A causally precedes B.

Session advancement is exactly this:

- Each session operation increments the session "position" by one
- Each send/recv on a channel is an event on that channel's timeline

- The session type's continuation depth IS the Lamport timestamp for that channel
- For multiple channels, the per-channel depths form a vector clock

```
Channel a: !String . ?Int . end          → positions: [0, 1, 2]
Channel b: ?Bool . !Nat . !Nat . end     → positions: [0, 1, 2, 3]

Vector clock at any program point:
  after send(a), recv(b):  {a: 1, b: 1}
  after recv(a), send(b):  {a: 2, b: 2}
  etc.
```

This vector clock is not imposed externally — it's *derived* from the session type itself. The session type IS the causal structure.

5.3 5c. Process Structure Determines the Partial Order

For a process with multiple channels:

```
proc a b =
  send "hello" a
  recv x b
  send x a
  stop
```

The session types determine the causal ordering:

- `send "hello" a` is at position `{a:0}`
- `recv x b` is at position `{a:0, b:0}` — concurrent with the send on `a` unless sequentially composed
- `send x a` is at position `{a:1}` — causally after the first send on `a`
- `stop` is at position `{a:2, b:1}` — causally after everything

But the *process structure* (sequential composition) adds more ordering: the `recv x b` must happen before `send x a` because `x` flows from recv to send. This data dependency is captured by the process AST, not by the session types alone.

Key observation: Session types provide the *per-channel* causal order. The process structure provides the *cross-channel* causal order. Together, they give the complete happens-before relation for effects.

5.4 5d. The Effect Handler

With effect positions derived from session advancement:

```
Effect Execution Protocol:
1. Compile process → propagator network with session cells + effect position cells
2. Run session advancement propagators to quiescence
   (session cells advance, effect positions update via  bridge)
3. After quiescence: read all effect position cells
4. Sort effects by position (total order within a channel, partial order across channel
5. Execute effects in position order
6. For concurrent effects (incomparable positions): execute in any order
   (they're on independent channels, so any interleaving is valid)
```

Step 6 is the crucial insight: *truly concurrent effects don't need ordering.* Effects on different channels can execute in any order because they're on independent resources (different files, different ports). The session type system's duality guarantee ensures that the two endpoints of a channel agree on the ordering, and the independence of separate channels ensures no interference.

5.5 5e. Comparison with Architectures A-C

Property	A (Barriers)	B (Timestamps)
Ordering source	External (walk order)	External (syntactic position)
Soundness proof	By construction (sequential walk)	Needs verification
Concurrent effects	Not supported	Via vector timestamps
Effect-computation interleaving	None (batch)	At barriers
Modification to propagator core	None	New cell type
Session type integration	None	None

Architecture D's distinctive advantage: *the ordering is a theorem, not a design decision.* Given a sound session type system with session fidelity, the derived effect ordering is provably correct. No additional verification is needed.

6 6. Deeper Implications: The Parameterised Monad View

6.1 6a. Session Types as Indexed Pre/Post Conditions

The connection between session types and parameterised monads (Atkey 2009, Orchard & Yoshida 2016) reveals a deeper algebraic structure.

A parameterised monad `M s s a` represents a computation that:

- Starts in state `s` (precondition)
- Ends in state `s` (postcondition)
- Produces a value of type `a`

Sequential composition matches postcondition to precondition:

```
bind : M s s a → (a → M s s b) → M s s b
```

Session types are a natural instance of this structure:

```
send : M (!A.S) (S) A      - starts with send-A session, ends with S, produces A
recv : M (?A.S) (S) A      - starts with recv-A session, ends with S, receives A
close : M (end) () ()      - starts with end session, ends with ()
```

The session type IS the pre/post state. Sequential composition of session operations IS the bind of the parameterised monad. The ordering is enforced by the type system — you can't bind a `recv` after a `close` because the postcondition `()` doesn't match the precondition `?A.S`.

Sources:

- Parameterised Notions of Computation — Atkey (2009)
- Session Types with Linearity in Haskell — Orchard & Yoshida (2016)
- Unifying Graded and Parameterised Monads — Katsumata et al.

6.2 6b. The Effect Quantale Connection

Graded monads generalize monads to track effects algebraically. The effect annotation forms a monoid (with sequential composition) and a partial order (with effect subsumption):

```
return : a → G_ a                      (pure computation, identity effect)
bind : G_i a → (a → G_j b) → G_{ij} b  (composition, effects combine)
```

When the effect monoid has a partial order that interacts well with composition, we get an *effect quantale* — an algebraic structure where:

- Effects form a lattice (partial order with joins)
- Sequential composition () distributes over joins
- The ordering respects composition: if `i  i'` and `j  j'`, then `i  j  i'  j'`

Session types, viewed as effect annotations, form an effect quantale:

- The session lattice provides the partial order
- Session continuation is sequential composition: `(!A.S)  (?B.T) = !A.?B.T` (compose the first session's operation with the second session's protocol)
- The ordering respects composition (monotonicity of session lattice merge)

This is the formal structure underlying the Galois connection. The session lattice and the effect position lattice are both effect quantales, and the Galois connection (,) is a quantale morphism — it preserves both the ordering and the sequential composition.

Sources:

- Effect Systems Revisited — Control-Flow Algebra and Semantics — Katsumata (2014)
- Polymorphic Iterable Sequential Effect Systems — Gordon (2021)

6.3 6c. The Category-Graded Monad Unification

Recent work by Katsumata, Orchard et al. unifies graded monads (effect systems) and parameterised monads (program logics, session types) into a single framework: *category-graded monads*.

In this framework, a category-graded monad is indexed by morphisms from a small category C. When C is a discrete category (objects only, no arrows), it degenerates to a graded monad. When C is the category of states and transitions, it degenerates to a parameterised monad.

Session types correspond to the case where C is the category of session states (session types are objects, session operations are morphisms). Effect annotations correspond to the case where C is the effect monoid.

The Galois connection between sessions and effects is, categorically, a functor between these two indexing categories — mapping session transitions to effect compositions while preserving the categorical structure.

Source:

- Unifying Graded and Parameterised Monads

7 7. Interaction with Our Existing Infrastructure

7.1 7a. The Cross-Domain Bridge Pattern

We already have Galois connections in the propagator network:

- Type lattice Session lattice bridge (`session-propagators.rkt`, S4e)
- Type lattice Interval lattice bridge (from the Galois connections work)

Adding a Session lattice Effect Position lattice bridge follows the same pattern:

```
;; effect-position-bridge.rkt (hypothetical)

;; Bridge propagator: session cell → effect position cell
(define (add-session-effect-bridge net sess-cell effect-cell channel-path)
  (net-add-propagator net
    (list sess-cell)
    (list effect-cell)
    (lambda (n)
      (define sess-val (net-cell-read n sess-cell))
      (cond
        [(sess-bot? sess-val) n]
        [else
         (define depth (session-depth sess-val))
         (net-cell-write n effect-cell
           (eff-pos channel-path depth))]))))

;; session-depth: count continuation nesting
(define (session-depth s)
  (match s
    [(sess-send _ cont) (add1 (session-depth cont))]
    [(sess-recv _ cont) (add1 (session-depth cont))]
    [(sess-dsend _ cont) (add1 (session-depth cont))]
```

```
[(sess-drecv _ cont) (add1 (session-depth cont))]
[(sess-end) 0]
[(sess-mu body) (session-depth body)]
[_ 0]))
```

7.2 7b. Integration with compile-live-process

Today's `compile-live-process` walks the AST sequentially and executes IO inline. With Architecture D, it would:

1. Walk the AST as before, installing session advancement propagators
2. Additionally install session-effect bridge propagators
3. Collect effect descriptors (without executing them)
4. After quiescence, execute effects in position order

The session advancement propagators already exist. The bridge propagators are new but follow an established pattern. The effect collection and execution are the novel components.

7.3 7c. The ATMS Connection for Speculative Effects

With session-derived positions, speculative effects become well-defined:

- The ATMS tracks alternative worldviews (§5.4 of the logic engine design)
- Each worldview has its own session advancement (different choices lead to different session states)
- Effect positions in one worldview may differ from positions in another
- Effects are only executed for the *chosen* worldview (after consistency filtering)

This connects to the research question §9d from the effectful propagators document: the ATMS enables hypothetical effects, and session-derived positions provide the ordering within each hypothesis.

8 8. The Session-Effect Duality

8.1 8a. An Observation

The duality operator on session types (`dual`) swaps send recv, choice offer. What happens to effect positions under duality?

```
Session:     !String . ?Int . end
Dual:        ?String . !Int . end

Positions:   [0: write, 1: read, 2: close]
Dual pos:    [0: read,  1: write, 2: close]
```

The dual session has the *same positions* but *different effects* at each position. This is exactly right: the two endpoints of a channel perform complementary effects at the same logical moments.

Effect duality: if endpoint A writes at position 0, endpoint B reads at position 0. The positions are synchronized (they're the same logical clock), but the effects are dual.

8.2 8b. Implications for IO

For IO channels (where one endpoint is the file system), the dual of the process's session type is the "file's session type" — what the file expects. If the process sends a string (write), the file receives a string (append to file). If the process receives (read), the file sends (provide file contents).

The IO bridge propagator already implements this duality implicitly: it watches for `sess-send` to know when to write and `sess-recv` to know when to read. Architecture D would make this correspondence explicit through the effect position lattice.

9 9. Challenges and Open Questions

9.1 9a. Recursive Sessions and Infinite Timelines

Recursive session types (`sess-mu`) produce potentially infinite causal timelines. The position counter would grow without bound:

```
t. !Nat . t       → positions: [0, 1, 2, 3, ...]
```

This doesn't break the Galois connection (positions still form a well-ordered chain), but it means:

- Effect cells can't be allocated statically for all positions
- The bridge propagator must create fresh effect cells during unfolding
- Widening may be needed if the effect lattice has infinite ascending chains

One approach: use *modular positions* — positions modulo the recursion depth. The recursive session `t. !Nat . t` has period 1, so positions 0, 1, 2, ... all have the same structure. The effect handler needs only the current and next positions, not the entire history.

9.2 9b. Choice and Non-Deterministic Sessions

Session choice (`sess-choice`) introduces branching in the causal timeline:

```
{left: !String . end, right: !Int . end}
```

After the choice, the timeline follows one branch. Effect positions on unchosen branches are unreachable.

This connects to the ATMS: each choice branch is a hypothesis. Effect positions in each branch are well-defined; the chosen branch's effects are executed; the unchosen branch's effects are discarded (nogoods prevent their execution).

9.3 9c. Data Dependencies Across Channels

Session types capture per-channel ordering but not cross-channel data flow. Consider:

```
recv x a            → position {a: 0}
send [f x] b        → position {b: 0}
```

Here `x` flows from channel `a` to channel `b`. The session types on `a` and `b` are independent, but the data dependency creates a cross-channel ordering constraint: `recv x a` must happen before `send [f x] b`.

This ordering is captured by the process structure (sequential composition in the AST), not by the session types. Architecture D captures it through the vector clock: after processing the recv, the vector clock is `{a:1, b:0}`. After the send, it's `{a:1, b:1}`. The happens-before relation `{a:1, b:0} < {a:1, b:1}` is correct.

But the propagator network doesn't see this data dependency unless it's encoded explicitly. The process AST walk provides it (it processes recv before send because they're sequentially composed). Whether the propagator network can capture this without the walk is the key question for a fully propagator-native implementation.

9.4 9d. When Sessions Don't Cover All Effects

Not all effects are associated with session channels. Top-level `eval` expressions with `SysCap` perform IO outside of any session. These effects have no session type to derive ordering from.

For these cases, Architecture D falls back to Architecture A (barriers) or Architecture B (timestamps). The session-derived ordering handles session-bound effects; unsessioned effects use the walk-order fallback.

This is acceptable — the session type system is designed for structured IO, and unstructured IO (top-level eval with SysCap) is intentionally less disciplined. The type system correctly reflects this: session-typed IO has stronger guarantees than SysCap IO.

10 10. Discussion: Resolving the Open Questions

This section captures findings from the Mar 6 design discussion, where the open questions from §9 were subjected to deeper analysis. Several key insights emerged that resolve the cross-channel data dependency problem (§9c), establish the architecture decision (§9d), and reveal a deep structural parallel between the `rel` and `proc` engines.

10.1 10a. Cross-Channel Data Dependencies Are Constraint Resolution (§9c Resolved)

The central open question of §9c was: can the propagator network capture cross-channel data dependencies *without* the sequential AST walk? The answer is *yes* — via *transitive closure of ordering edges*, which is a *monotone fixed-point computation* and therefore lives natively in the propagator network.

The insight: there are two kinds of ordering edges in a process:

1. *Session ordering edges* (per-channel): `send a` → `recv a` → `send a` (from session type continuation structure)

2. *Data-flow edges* (cross-channel): `recv x a` → `send [f x] b` (from variable binding — `x` is produced by the recv and consumed by the send)

The *complete ordering* is the transitive closure of the union of these edges. And transitive closure is a monotone operation on sets of edges (adding edges never removes existing ordering relationships). This means a propagator can incrementally compute the ordering:

```
Ordering Cell: {(a < a), (b < b)}        ;; initial: session edges only
    + data-flow edge (a < b)                ;; recv x a flows to send [f x] b
    → {(a < a), (b < b), (a < b)}
    + transitive closure
    → {(a < a), (b < b), (a < b), (a < b)}  ;; a before b via transitivity
```

Each new edge makes the ordering set "larger" (monotone). The propagator reaches a fixed point when no new transitive edges can be derived. At that point, the partial order is complete, and effects can be executed in any linearization of the partial order (concurrent effects in any interleaving).

This is the same computational pattern as `rel`: the logic engine computes the transitive closure of logical dependencies via propagation. Here, `proc` computes the transitive closure of causal dependencies via the same mechanism.

10.2 10b. ATMS for Branching Effect Orders

When a process contains `proc-case` (branching based on received values), each branch may create different data-flow patterns and therefore different ordering edges:

```
proc a b =
  recv x a
  case x of
    | "hello" -> send "world" b     ;; branch 1: edge (a < b)
    | "query" -> send [lookup x] b  ;; branch 2: edge (a < b) + data dep on x
  stop
```

The ATMS handles this naturally. Each branch is a *hypothesis* (an ATMS assumption). Ordering edges derived from a branch are tagged with that branch's assumption. The ATMS maintains multiple consistent worldviews:

```
Worldview 1 (x = "hello"): ordering = {(a < b)}
Worldview 2 (x = "query"): ordering = {(a < b), (a depends-on x)}
```

When the recv resolves (the value of `x` becomes known), one worldview is selected and the other becomes a nogood. Effects are executed according to the ordering in the chosen worldview. This is exactly how the logic engine handles `amb` — maintain hypotheses, resolve when evidence arrives, discard inconsistent worldviews.

10.3 10c. The `rel=/=proc` Structural Parallel

The pattern that emerges reveals a deep structural parallel between our two computational engines:

Dimension	`rel` (Logic Engine)	`proc` (Session Runtime)
Variables	Logic variables (unification)	Session cells (protocol state)
Constraints	Equality constraints (type/term)	Ordering constraints (causal)
Propagation	Unification propagators	Session advancement + ordering propa
Fixed point	Constraint closure (all equalities derived)	Ordering closure (transitive closure of
Hypotheses	ATMS worldviews (choice points)	ATMS worldviews (branch alternatives
Resolution	Evidence selects worldview	Received value selects worldview
Execution	Proof terms (constructive evidence)	Effects (IO operations in causal order)
Substrate	Propagator network + ATMS + stratification	Propagator network + ATMS + effect

Both engines share the three-layer architecture from the Logic Engine Design:

```
Layer 1: Propagator Network   - monotone fixed points on lattice cells
Layer 2: ATMS                 - hypothetical reasoning over alternatives
Layer 3: Control              - stratification (rel) / effect handler (proc)
```

The Logic Engine Design (`2026-02-24_LOGIC_ENGINE_DESIGN.org`, §5) established this three-layer pattern for recovering non-monotone behavior (negation, choice points) on a monotone substrate. The same pattern recovers non-monotone behavior (effect ordering, sequential execution) for `proc`. **The Layered Recovery Principle is general**: it applies to any domain where non-monotone operations must be performed on convergent infrastructure.

10.4 10d. Partial Order Planning Connection

The ordering computation described in §10a is isomorphic to **Partial Order Planning (POP)**, a classical AI planning technique:

POP Concept	Effect Ordering Analogue
Actions	IO effects (read, write, open, close)
Causal links	Data-flow edges (variable flows from one effect to another)
Ordering constraints	Session ordering edges + data-flow edges
Initial constraints	Session type structure (per-channel total order)
Plan alternatives	`proc-case` branches
Threats	Potential re-orderings that violate data dependencies
Threat resolution	Transitive closure forces ordering; ATMS prevents inconsistent orderings
Least commitment	Don't order concurrent effects unnecessarily

The POP connection is significant because it confirms that *least commitment* is the correct strategy: effects that are truly concurrent (different channels, no data dependencies) should NOT be forced into an arbitrary order. The partial order is the maximally concurrent schedule. Any linearization is valid for execution.

This also connects to the CALM theorem: the ordering computation is monotone (transitive closure, edge accumulation), so it can proceed without coordination. The only non-monotone operation is effect *execution* (which consumes the plan and produces side effects), and that happens at the control layer boundary — exactly where the Layered Recovery Principle predicts it should be.

10.5 10e. Architecture D Refined: Five Layers

With the cross-channel resolution, the architecture extends to five layers:

```
Layer 1: Session Advancement            [exists - session-runtime.rkt]
    Session cells advance monotonically through session types.
    Each advancement is a causal clock tick for that channel.

Layer 2: Data-Flow Analysis             [NEW - per-variable ordering edges]
    Analyze variable bindings to derive cross-channel ordering edges.
    recv x a → send [f x] b    ordering edge (a_pos < b_pos)

Layer 3: Transitive Closure             [NEW - ordering cell propagator]
```

```
    Compute the complete partial order by propagating ordering edges
    to their transitive closure. Monotone fixed-point computation.
    Concurrent effects remain unordered (least commitment).

Layer 4: ATMS Branching                  [exists - atms.rkt]
    Maintain per-branch ordering hypotheses for proc-case.
    Resolve when branch is chosen; discard inconsistent worldviews.

Layer 5: Effect Handler                  [NEW - sequential execution]
    Read the resolved partial order. Execute effects in any valid
    linearization. This is the only non-monotone step (actual IO).
    Equivalent to the "stratification barrier" in the logic engine.
```

Layers 1-4 are monotone and live entirely within the propagator network. Layer 5 is the control boundary — the point where monotone reasoning hands off to sequential execution, just as the logic engine's stratification barrier hands off to negation evaluation.

10.6 10f. `main` as Implicit Session

An important observation: `main` is already an implicit session. Today, `main` is a function that executes IO effects in sequential order. This is exactly what Architecture A (Stratified Barriers) provides — and it's also what Architecture D provides for a single-channel, single-session process.

In fact, *Architecture A is a degenerate case of Architecture D*:

```
Architecture A: main is a sequential walk → effects in walk order
Architecture D: main as implicit single-channel session → effects in session order
                (and walk order = session order for the degenerate case)
```

This observation means the transition from A to D is smooth. The current behavior of `main` (sequential IO execution via AST walk) is exactly what D would produce for a process with a single implicit session. As `main` gains explicit session structure (multi-channel, concurrent sub-processes), D naturally extends to provide the richer ordering semantics.

For non-`main` top-level IO (REPL, `eval` with `SysCap`), there is no session type, so Architecture A remains the fallback. This is correct — unstructured IO in the REPL is intentionally less disciplined. The type system makes this distinction visible.

11 11. Architecture Decision: A + D

11.1 11a. The Recommended Pair

Architecture D provides session-derived ordering for session-typed IO (the structured case). Architecture A provides walk-based ordering for unsessioned IO (the unstructured case). Together, they cover all IO scenarios:

Scenario	Architecture	Ordering Source	Guarantees
Session-typed IO (`defproc`)	D	Session type structure	Session fideli
Multi-channel IO	D	Vector clock from per-channel sessions	Concurrent e
`main` (single-channel)	D (degenerate)	Implicit session walk order	Same as curr
REPL / top-level `eval`	A	AST walk order	Sequential ex
Non-IO pure computation	—	N/A	No effects to

11.2 11b. Why B Is Subsumed

Architecture B (Timestamped Effect Cells) assigns syntactic timestamps to effects and executes in timestamp order at barriers. For session-typed effects, Architecture D provides strictly more information:

- *D's ordering is intrinsic*: derived from the session type, not from syntactic position. Refactoring code doesn't change the ordering (it can't — the session type is invariant).
- *D's ordering is verified*: session fidelity is a theorem of the type system. Timestamp assignment has no such guarantee.
- *D handles concurrency*: vector clocks from multi-channel sessions naturally express concurrent effects. Timestamps require explicit vector-timestamp machinery.
- *D is compositional*: session type composition ($\text{PROTOCOLS}_{\text{ASTYPES.org}}$) composes effect orderings automatically.

For unsessioned effects, B and A are equivalent (both impose external ordering). A is simpler (walk order requires no new infrastructure). Therefore B adds nothing that A + D don't already provide.

11.3 11c. When C Might Be Relevant

Architecture C (Reactive Effect Streams with topological scheduling) could be relevant for effects that are neither session-typed nor sequentially walked — e.g., declarative effect specifications in a constraint language. This remains a research direction but is not needed for Phase 0.

11.4 11d. Layered Architecture

```
Layer 0: Propagator Network (monotone fixed points)                [exists]
Layer 1: Session Lattice (protocol verification)                   [exists]
Layer 2: Effect Position Lattice (session-derived ordering)        [NEW - D]
Layer 3: ATMS (hypothetical effect orderings for branches)         [exists]
Layer 4: Effect Handler (sequential execution in position order)   [NEW - D]
Fallback: AST Walk (for unsessioned effects)                       [exists - A]
```

12 12. Formalization Roadmap

If we pursue Architecture D, the implementation would proceed:

12.1 Phase D1: Effect Position Lattice (Small, Self-Contained)

- Define `eff-bot`, `eff-pos(path, depth)`, `eff-top`
- Implement `eff-merge`, `eff-contradicts?`
- Tests: lattice properties (bot identity, idempotent, top absorption)

12.2 Phase D2: Session-Effect Bridge Propagator

- Implement (session → effect position) and (effect position → session)
- Bridge propagator: watches session cell, updates effect cell
- Tests: bridge fires correctly on session advancement

12.3 Phase D3: Effect Descriptor Cells

- Define effect descriptors: `eff-write(data)`, `eff-read`, `eff-close`, `eff-open(path, mode)`
- Effect descriptor cells with position + descriptor
- Tests: descriptors accumulate monotonically

12.4 Phase D4: Effect Handler

- Read all effect descriptor cells after quiescence
- Sort by position (total within channel, partial across channels)
- Execute in order, feeding results back
- Tests: end-to-end IO through session-derived ordering

12.5 Phase D5: Vector Clock for Multi-Channel

- Extend positions to vectors for multi-channel processes
- Partial order for concurrent effects
- Tests: concurrent channel effects in correct order

13 13. Critique and Limitations

13.1 13a. Data-Flow Analysis Complexity

The cross-channel data-flow analysis (§10a) requires tracking variable bindings through process structure. For simple cases (variable flows directly from recv to send), this is straightforward. For complex cases (pattern matching on received values, function application, higher-order bindings), the data-flow graph grows in complexity.

However, the process language (`proc`) is intentionally restricted compared to the expression language. Processes don't have arbitrary higher-order functions — they have `send`, `recv`, `proc-case`, `proc-par`, and sequential composition. This restriction bounds the data-flow analysis to a manageable set of cases.

13.2 13b. Cycle Detection as Deadlock Detection

If the transitive closure computation discovers a cycle in the ordering graph (position A $<$ position B $<$ position A), this indicates a *deadlock* — two effects that each depend on the other having already executed. This is not a failure of the ordering algorithm; it's a detection of a genuine program error.

The session type system already prevents many deadlocks (session fidelity ensures that endpoints agree on ordering). But cross-channel cycles can still arise:

```
Process 1: recv x a . send [f x] b
Process 2: recv y b . send [g y] a
```

Each process waits for the other. The transitive closure would produce `a < b < a` — a cycle. The ordering propagator can detect this and raise an error at compile time, turning a runtime deadlock into a compile-time rejection. This is a significant advantage — the ordering analysis serves double duty as a static deadlock detector for cross-channel data dependencies.

13.3 13c. Performance Considerations

Transitive closure on a set of ordering edges has $O(n^3)$ worst-case complexity (Floyd-Warshall). For typical process structures, `n` is the number of IO operations in a single process — usually small (< 50). The propagator-based incremental computation may be more efficient than batch algorithms because edges arrive incrementally as session cells advance.

For very large processes with many channels, the vector clock representation grows linearly with the number of channels. This is the same scaling behavior as distributed vector clocks, and the same mitigation applies: only track channels that interact (sparse vector clocks).

13.4 13d. The "Fully Native" Limit

Even with all five layers, the effect handler (Layer 5) remains non-monotone — it executes actual IO operations that produce observable side effects. This is irreducible: the CALM theorem guarantees that non-monotonic operations (which include all observable effects) require coordination.

The achievement is not eliminating the coordination point, but *minimizing* it. All reasoning about ordering is done monotonically in the propagator network. The only coordination point is the final execution barrier. This is optimal — it's the minimum coordination required by the CALM theorem.

14 14. Conclusions

14.1 14a. The Core Discovery

Session types are not just protocol specifications — they are *causal clocks*. The continuation structure of a session type encodes a total order on effects per channel, and the independence of separate session types encodes concurrency. Together, they provide a vector-clock-like causal ordering that is:

1. *Intrinsic*: Derived from the session type, not imposed externally
2. *Sound*: Follows from session fidelity (a theorem, not a design choice)
3. *Compositional*: Multiple channels compose via vector clocks
4. *Compatible with monotone propagation*: Session advancement is monotone; effect position derivation via is monotone; both live happily in the propagator network

14.2 14b. The Galois Connection

The Galois connection `(, )` between the session lattice and the effect position lattice formalizes this relationship:

- extracts ordering information from session types
- reconstructs session context from ordering information
- The adjunction guarantees soundness: session fidelity implies effect ordering

This connection is a quantale morphism between two effect quantales, reflecting the deep algebraic structure shared by session types and effect systems.

14.3 14c. Architecture A + D

The recommended architecture pairs session-derived ordering (D) with walk-based barriers (A) as a fallback:

- *D* handles session-typed IO — the structured, common case. Ordering is a theorem from session fidelity, not a design choice. Cross-channel data dependencies are resolved by transitive closure (a monotone fixed point). Branch alternatives are maintained by the ATMS.
- *A* handles unsessioned IO — the REPL, top-level eval, `main` without explicit sessions. Walk order provides sequential execution.
- *B* is subsumed — it adds nothing that A + D don't provide.

14.4 14d. The `rel=/=proc` Parallel

The deep structural parallel between the logic engine (`rel`) and the session runtime (`proc`) confirms that the Layered Recovery Principle is general: non-monotone behavior is recovered on a monotone substrate by inserting control layers at the boundaries of monotone phases. The propagator network is the shared foundation; the ATMS provides hypothetical reasoning; the control layer (stratification for `rel`, effect handler for `proc`) performs the non-monotone operations at precisely the points where coordination is required.

14.5 14e. Architecture D as Natural Extension

Architecture D emerges as a natural extension of our existing infrastructure:

- The session lattice already exists
- Session advancement propagators already exist
- Cross-domain bridge propagators are an established pattern
- The ATMS for branching already exists
- The effect position lattice is a simple new lattice
- The bridge propagator follows the existing Galois connection pattern
- Transitive closure is a standard propagator computation

The novel contributions are:

1. Recognizing that session types provide causal clocks intrinsically
2. Formalizing the session-to-effect mapping as a Galois connection
3. Using transitive closure of ordering edges for cross-channel dependencies
4. Leveraging the ATMS for branching effect orders
5. Identifying the `rel=/=proc` structural parallel

15 References

15.1 Prologos Internal

- Effectful Propagators Research: `docs/tracking/2026-03-06_EFFECTFUL_PROPAGATORS_RESEARCH`
- IO Implementation PIR: `docs/tracking/2026-03-06_IO_IMPLEMENTATION_PIR.md`
- Galois Connections & AI: `docs/tracking/2026-02-27_1026_GALOIS_CONNECTIONS_ABSTRACT_IN`
- Logic Engine Design: `docs/tracking/2026-02-24_LOGIC_ENGINE_DESIGN.org`
- Session Type Design: `docs/tracking/2026-03-03_SESSION_TYPE_DESIGN.md`
- Session Type PIR: `docs/tracking/2026-03-04_SESSION_TYPE_PIR.md`
- Protocols as Types: `docs/tracking/principles/PROTOCOLS_AS_TYPES.org`
- Effectful Computation on Propagators: `docs/tracking/principles/EFFECTFUL_COMPUTATION_ON`

15.2 Foundational

- Propositions as Sessions — Wadler (ICFP 2012)
- Linear Logic Propositions as Session Types — Caires & Pfenning (MSCS 2012)
- Session Types as Intuitionistic Linear Propositions — Caires & Pfenning (CONCUR 2010)
- Cut Reduction as Asynchronous Session-Typed Communication — DeYoung, Caires, Pfenning, Toninho (CSL 2012)

15.3 Causality and Linear Logic

- Causality in Linear Logic — Hayman & Winskel (FoSSaCS 2019)
- Games and Full Completeness for MLL — Abramsky & Jagadeesan
- Sequentiality vs. Concurrency in Games and Logic — Melliès

15.4 Effect Systems and Graded Monads

- Parameterised Notions of Computation — Atkey (JFP 2009)
- Session Types with Linearity in Haskell — Orchard & Yoshida (2016)
- Unifying Graded and Parameterised Monads — Katsumata, Orchard et al.
- Effect Systems Revisited — Katsumata (2014)
- Polymorphic Iterable Sequential Effect Systems — Gordon (TOPLAS 2021)
- Data-Flow Analyses as Effects and Graded Monads — Orchard et al.

15.5 Session Type Properties

- Comparing Type Systems for Deadlock Freedom — Kokke et al.
- Manifest Deadlock-Freedom for Shared Session Types — Balzer et al.
- Deadlock-Free Session Types in Linear Haskell — Kokke (2021)

15.6 Propagators, CRDTs, and Distributed Systems

- Revised Report on the Propagator Model — Radul & Sussman
- CRDTs (Wikipedia)
- Vector Clocks (Wikipedia)
- Kmett's Propagators (GitHub)
- CALM Theorem — Hellerstein (CIDR 2011)

15.7 Planning and Ordering

- Weld, D.S. — An Introduction to Least Commitment Planning (AI Magazine, 1994)
- Penberthy, J.S. & Weld, D.S. — UCPOP: A Sound, Complete, Partial Order Planner for ADL (KR 1992)